Requirement: โปรเจกต์ฝึกงาน เว็บไซต์ STARTER (Mini Project)

Oct 16, 2026 · @sutiwat.p

1. ภาพรวมโปรเจกต์

แต่ละคนสร้างเว็บไซต์ธุรกิจ 1 เว็บแบบครบวงจร (หน้าบ้าน + หลังบ้าน + ฐานข้อมูล) ให้ได้ตามสเปกแพ็กเกจ **STARTER** ภายใน **10 วันทำงาน (ยืดหยุ่น)** ทำคนเดียว

เป้าหมายคือให้น้องได้ทำงานเหมือนรับงานลูกค้าจริง ไม่ใช่แค่ฝึกเขียนโค้ด ประเภทธุรกิจน้องเลือกเอง แต่สเปกขั้นต่ำต้องครบตามเอกสารนี้

สิ่งที่น้องจะได้ต่อยอดจาก แบบฝึกแรก

- จากตารางเดียว → หลายตารางที่มีความสัมพันธ์กัน
- จาก CRUD ธรรมดา → มีระบบ login แยกสิทธิ์ admin
- จากหน้าเดียว → หลายหน้า มี routing และ SEO
- จาก Prisma → Drizzle ORM (ได้เห็นว่า ORM คนละตัวคิดต่างกันยังไง)
- จากงานทดลอง → งานที่ต้องส่งมอบให้ "ลูกค้า" ใช้ต่อได้จริง

2. ขอบเขตงานและการเลือกธุรกิจ

ขั้นแรก ต้องเลือกธุรกิจสมมติ 1 ประเภท แล้วเขียน brief สั้นๆ ส่งพี่เลี้ยงก่อนเริ่มโค้ด (วันที่ 1) เช่น ร้านกาแฟ, คลินิกทำฟัน, ร้านซ่อมมือถือ, สตูดิโอถ่ายภาพ, โฮมสเตย์, ร้านเช่าชุด

Brief ต้องมี: ชื่อแบรนด์, กลุ่มลูกค้า, สีหลัก 2-3 สี, โลโก้ (ทำเองง่ายๆ ได้), สินค้า/บริการอย่างน้อย 6 รายการ

ข้อห้าม: ห้ามเลือกธุรกิจที่ต้องมีระบบตะกร้าสินค้า/ชำระเงิน เพราะเกินแพ็กเกจ STARTER

ฟีเจอร์ตามแพ็กเกจ → สิ่งที่น้องต้องทำจริง

ฟีเจอร์ในแพ็กเกจ	ความหมายในโปรเจกต์นี้
ออกแบบตามแบรนด์	ใช้สี/ฟอนต์/โลโก้ตาม brief ทั้งเว็บ ตั้งค่าใน Tailwind theme
สูงสุด 3 หน้า	หน้า Home, About/Services, Contact (หน้า admin ไม่นับ)
Responsive	ใช้งานได้ที่ 375px, 768px, 1280px
ระบบจัดการเนื้อหา	admin แก้ข้อมูลเว็บได้เองโดยไม่ต้องแก้โค้ด
Contact Form	ฟอร์มส่งข้อความ บันทึกลง DB และดูได้ในหลังบ้าน
LINE / Social Media	ลิงก์/ปุ่ม LINE, Facebook, IG ที่ admin แก้ได้
Basic SEO	title, meta description, OG tags ต่อหน้า แก้ได้จากหลังบ้าน
แก้ไขงาน 2 ครั้ง	พี่เลี้ยงจะ review 2 รอบ (วันที่ 5 และ 9) น้องต้องแก้ตาม

3. Tech stack และโครงสร้างโปรเจกต์

ส่วน	เครื่องมือ
Frontend	React + TypeScript (Vite), Tailwind CSS, React Router
Backend	Bun + Hono
ORM	Drizzle ORM + drizzle-kit (migration)
Validation	Zod (ใช้ทั้ง backend และแชร์ type ให้ frontend)
Database	PostgreSQL, จัดการผ่าน pgAdmin
Auth	JWT เก็บใน httpOnly cookie, hash รหัสผ่านด้วย `Bun.password`

โครงสร้างโฟลเดอร์ขั้นต่ำ (แยก client / server เหมือนโปรเจกต์ Todo)

```
project/
├─ client/
│  └─ src/
│     ├─ pages/          # หน้าเว็บ public + admin
│     ├─ components/
│     ├─ api/            # ตัวกลางเรียก backend ทั้งหมด
│     └─ types/
├─ server/
│  ├─ src/
│  │  ├─ index.ts        # entry point
│  │  ├─ routes/         # แยกไฟล์ตาม resource
│  │  ├─ db/
│  │  │  ├─ schema.ts    # Drizzle schema
│  │  │  └─ index.ts     # db connection
│  │  ├─ validators/     # Zod schemas
│  │  └─ middleware/     # auth guard, error handler
│  ├─ drizzle/           # ไฟล์ migration
│  └─ .env.example
└─ README.md
```

กติกา: ห้าม commit ไฟล์ `.env` ต้องมี `.env.example` บอกว่าต้องตั้งค่าอะไรบ้าง และต้องมี seed script สร้าง admin คนแรก

4. Requirement หน้าบ้าน (Public)

ทุกข้อความ รูป และรายการบนหน้าบ้าน **ต้องดึงจาก API** ห้าม hardcode ในโค้ด (ยกเว้นข้อความ UI เช่น ปุ่ม "ส่ง")

ส่วนที่ใช้ร่วมทุกหน้า

- Navbar: โลโก้, เมนู 3 หน้า, ไฮไลต์หน้าที่อยู่, มือถือเป็น hamburger menu
- Footer: ที่อยู่, เบอร์โทร, เวลาทำการ, ไอคอน social
- ปุ่ม LINE ลอยมุมขวาล่าง (floating) ลิงก์ตามที่ตั้งในหลังบ้าน

หน้า 1: Home `/`

- Hero: หัวข้อใหญ่ ข้อความรอง รูปพื้นหลัง ปุ่ม call-to-action ไปหน้า Contact
- แสดงบริการ/สินค้าเด่น 3 รายการ (ที่ admin ติ๊ก "แสดงหน้าแรก")
- จุดเด่นของร้าน 3-4 ข้อ (ไอคอน + ข้อความ)

หน้า 2: About / Services `/about`

- เกี่ยวกับร้าน: เนื้อหายาว + รูป
- รายการบริการ/สินค้าทั้งหมด แสดงเป็น card: รูป, ชื่อ, รายละเอียดสั้น, ราคา (ถ้ามี)
- กรองตามหมวดหมู่ได้ (อย่างน้อย 2 หมวด)

หน้า 3: Contact `/contact`

- ฟอร์ม: ชื่อ*, เบอร์โทร*, อีเมล, หัวข้อ, ข้อความ* (* = บังคับ)
- validate ฝั่ง client ก่อนส่ง และแสดง error ใต้ช่องที่ผิด
- ระหว่างส่ง ปุ่มต้อง disable + แสดง loading, ส่งสำเร็จแสดงข้อความขอบคุณและล้างฟอร์ม
- แสดงข้อมูลติดต่อ + แผนที่ Google Maps (embed iframe จากลิงก์ที่ admin ตั้ง)

สถานะที่ต้องมีทุกหน้า

- Loading (skeleton หรือ spinner) ระหว่างรอ API
- Error เมื่อ API ล่ม ไม่ใช่หน้าขาว
- หน้า 404 เมื่อเข้า URL ที่ไม่มี

5. Requirement หลังบ้าน (Admin)

หลังบ้านอยู่ที่ `/admin` เข้าได้เฉพาะคนที่ login แล้ว ไม่ต้องสวยเท่าหน้าบ้าน แต่ต้องใช้งานง่ายสำหรับเจ้าของร้านที่ไม่รู้เรื่องโค้ด

5.1 Login

- `/admin/login` กรอกอีเมล + รหัสผ่าน
- ผิดแสดงข้อความ "อีเมลหรือรหัสผ่านไม่ถูกต้อง" (ห้ามบอกว่าผิดช่องไหน)
- เข้า `/admin/*` โดยไม่ login ต้องเด้งกลับหน้า login
- มีปุ่ม logout และ session หมดอายุใน 1 วัน

5.2 Dashboard `/admin`

- จำนวนข้อความติดต่อที่ยังไม่อ่าน
- จำนวนบริการ/สินค้าทั้งหมด
- ข้อความติดต่อล่าสุด 5 รายการ

5.3 ตั้งค่าเว็บไซต์ `/admin/settings`

แก้ได้ในฟอร์มเดียว: ชื่อร้าน, โลโก้ (URL), เบอร์โทร, อีเมล, ที่อยู่, เวลาทำการ, ลิงก์ LINE / Facebook / IG, ลิงก์ Google Maps

5.4 จัดการเนื้อหาแต่ละหน้า `/admin/pages`

- แก้ข้อความ Hero, จุดเด่น, เนื้อหา About
- แก้ SEO ต่อหน้า: title, meta description, OG image

5.5 จัดการบริการ/สินค้า `/admin/services`

- ตารางรายการ: ค้นหาด้วยชื่อ, กรองตามหมวด, แบ่งหน้า 10 รายการ
- เพิ่ม / แก้ไข / ลบ (ลบต้องถามยืนยันก่อน)
- ฟิลด์: ชื่อ, หมวดหมู่, รายละเอียด, ราคา, รูป, แสดงหน้าแรก (ใช่/ไม่), เปิด/ปิดการแสดงผล, ลำดับการแสดง
- จัดการหมวดหมู่ได้ (เพิ่ม/แก้/ลบ) ลบหมวดที่ยังมีสินค้าอยู่ไม่ได้

5.6 ข้อความติดต่อ `/admin/messages`

- รายการข้อความ เรียงใหม่สุดก่อน ตัวที่ยังไม่อ่านเป็นตัวหนา

- กดเปิดดูแล้วเปลี่ยนเป็น "อ่านแล้ว" อัตโนมัติ
- ลบข้อความได้

5.7 รูปภาพ

รอบนี้ **ใช้ URL รูป** ไปก่อน (ฝากรูปที่อื่นแล้วเอาลิงก์มาใส่) ไม่ต้องทำระบบอัปโหลด เพราะ 10 วันไม่พอทำให้ดี ใครเสร็จเร็วไปทำในข้อ bonus

6. ฐานข้อมูลขั้นต่ำ

มีอย่างน้อย 6 ตาราง ทุกตารางต้องมี `id`, `created_at`, `updated_at` น้องเพิ่มฟิลด์ตามธุรกิจได้ แต่ห้ามตัดฟิลด์ข้างล่าง

ตาราง	ฟิลด์หลัก	หมายเหตุ
`admins`	email (unique), password_hash, name	ห้ามเก็บรหัสผ่านตรงๆ
`site_settings`	shop_name, logo_url, phone, email, address, opening_hours, line_url, facebook_url, instagram_url, map_url	มีแถวเดียวทั้งระบบ
`pages`	slug (unique: home/about/contact), seo_title, seo_description, og_image_url, content (jsonb)	content เก็บข้อความ hero/จุดเด่น/about
`categories`	name, slug (unique), sort_order	
`services`	category_id (FK), name, description, price (nullable), image_url, is_featured, is_active, sort_order	FK ห้ามลบ category ทิ้งถ้ายังมี service
`contact_messages`	name, phone, email, subject, message, is_read	

ทำไม `pages.content` เป็น jsonb: แต่ละหน้ามีโครงสร้างต่างกัน ถ้าแตกเป็นตารางย่อยจะเยอะเกินสำหรับ 3 หน้า แต่ต้องมี Zod schema คุม shape ของ jsonb ทุกครั้งที่บันทึก

ต้องส่ง: ไฟล์ `schema.ts`, migration ที่รันได้, seed script (admin 1 คน, หมวด 2, service 6, settings 1, pages 3) และรูป ER Diagram (export จาก pgAdmin หรือวาดเองก็ได้)

7. API Spec

ทุก endpoint ขึ้นต้นด้วย `/api` แบ่งเป็น public (ใครก็เรียกได้) กับ admin (ต้องมี cookie login)

Public

Method	Endpoint	ใช้ทำอะไร
GET	`/api/settings`	ข้อมูลร้าน (ใช้ใน navbar/footer)
GET	`/api/pages/:slug`	เนื้อหา + SEO ของหน้านั้น
GET	`/api/categories`	หมวดหมู่ทั้งหมด
GET	`/api/services?category=&featured=`	บริการที่เปิดแสดงผล
POST	`/api/contact`	ส่งข้อความติดต่อ

Admin

Method	Endpoint	ใช้ทำอะไร
POST	`/api/admin/login`	login ตั้ง cookie
POST	`/api/admin/logout`	ลบ cookie
GET	`/api/admin/me`	เช็คว่ายัง login อยู่ไหม
GET	`/api/admin/dashboard`	ตัวเลขสรุปหน้า dashboard
PUT	`/api/admin/settings`	แก้ข้อมูลร้าน
PUT	`/api/admin/pages/:slug`	แก้เนื้อหา/SEO
GET / POST	`/api/admin/categories`	ดู / เพิ่มหมวด
PATCH / DELETE	`/api/admin/categories/:id`	แก้ / ลบหมวด
GET / POST	`/api/admin/services?q=&category=&page=`	ดู (มี search + pagination) / เพิ่ม
PATCH / DELETE	`/api/admin/services/:id`	แก้ / ลบ
GET	`/api/admin/messages`	รายการข้อความ

Method	Endpoint	ใช้ทำอะไร
GET / DELETE	`/api/admin/messages/:id`	เปิดอ่าน (mark read) / ลบ

รูปแบบ response ต้องเหมือนกันทุก endpoint

```
// สำเร็จ
{ "success": true, "data": { } }

// list ที่มีแบ่งหน้า
{ "success": true, "data": [ ], "meta": { "page": 1, "limit": 10, "total": 42 } }

// ผิดพลาด
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "...",
"details": { } } }
```

Status code ที่ต้องใช้ให้ถูก: 200, 201 (สร้างใหม่), 400 (validate ไม่ผ่าน), 401 (ไม่ได้ login), 404 (ไม่พบ), 409 (ซ้ำ เช่น slug ซ้ำ, ลบหมวดที่มีสินค้า), 500

8. Non-functional Requirement

Responsive

- ทดสอบที่ 375px (มือถือ), 768px (แท็บเล็ต), 1280px (desktop) ไม่มี scroll แนวนอน
- ตาราง admin บนมือถือต้อง scroll ในกรอบได้ ไม่ล้นจอ

SEO พื้นฐาน

- แต่ละหน้าเปลี่ยน `<title>` และ meta description ตามข้อมูลจาก `pages`
- มี OG tags (title, description, image) ให้แชร์ลง LINE/Facebook แล้วขึ้นรูป
- รูปทุกรูปมี `alt`, ใช้ heading `h1` หน้าละ 1 ตัว
- มีไฟล์ `robots.txt` และ `sitemap.xml` (เขียนมือได้ เพราะมีแค่ 3 หน้า)

> ข้อควรรู้: *React SPA* ทำ *SEO* ได้จำกัด เพราะ *bot* บางตัวไม่รัน *JS* ระดับ *STARTER* ยอมรับได้ แต่น้องต้องอธิบายใน *README* ได้ว่าทำไม และถ้าจะแก้ต้องทำยังไง *(เช่น SSR/prerender)*

Security

- รหัสผ่าน hash ด้วย `Bun.password.hash` (argon2/bcrypt)
- JWT อยู่ใน cookie `httpOnly`, `sameSite=lax`, ห้ามเก็บใน localStorage
- ทุก endpoint `/api/admin/*` ผ่าน middleware เช็ค token
- Contact form จำกัด 5 ครั้ง / 10 นาที / IP กัน spam
- Error 500 ห้ามส่ง stack trace กลับไปให้ client

Validation

- ทุก request body/query ผ่าน Zod ที่ backend เสมอ (validate frontend อย่างเดียวไม่นับ)
- เบอร์โทร 9-10 หลัก, อีเมลถูกรูปแบบ, ข้อความไม่เกิน 1,000 ตัวอักษร

Code quality

- TypeScript ห้ามใช้ `any` ยกเว้นมีเหตุผลเขียนคอมเมนต์ไว้
- commit อย่างน้อยวันละ 1 ครั้ง ข้อความ commit อ่านรู้เรื่อง

9. Timeline 10 วันทำงาน

ลำดับคือ **ฐานข้อมูล → API → หน้าบ้าน → หลังบ้าน** เพราะน้องถนัด backend จาก Todo มาแล้ว ทำส่วนที่มั่นใจก่อนให้มีฐานแน่น

วัน	งาน	ต้องเสร็จ
1	เลือกธุรกิจ, เขียน brief, ร่าง wireframe 3 หน้า (กระดาษก็ได้), setup repo	brief ผ่านพี่เลี้ยง
2	Drizzle schema, migration, seed, ER diagram	`bun run seed` แล้วมีข้อมูลใน pgAdmin
3	Public API + Zod + response format	ทดสอบทุก endpoint ผ่าน Postman/Httpie
4	Auth (login/logout/middleware) + Admin API	เรียก admin API โดยไม่ login แล้วได้ 401
5	Layout, Navbar/Footer, หน้า Home	**Review รอบที่ 1** (โครงสร้าง DB + API)
6	หน้า About/Services + Contact form	ส่งฟอร์มแล้วข้อมูลเข้า DB
7	Admin: login, dashboard, settings, pages	แก้ข้อมูลร้านแล้วหน้าบ้านเปลี่ยน
8	Admin: services, categories, messages	CRUD ครบ
9	SEO, responsive, loading/error state	**Review รอบที่ 2** (ทั้งระบบ)
10	แก้ตาม review, README, demo	นำเสนอ 15 นาที

ถ้าวันไหนหลุด ห้ามข้ามไปทำต่อเงียบๆ ต้องแจ้งพี่เลี้ยงภายในวันนั้น

10. สิ่งที่ต้องส่งและเกณฑ์ประเมิน

สิ่งที่ต้องส่ง

- ☐ Git repo (client + server ใน repo เดียว)
- ☐ README: วิธีติดตั้ง, ตั้งค่า `.env`, รัน migration/seed, รันโปรเจกต์, บัญชี admin ทดสอบ
- ☐ ER Diagram
- ☐ ไฟล์ collection API (Postman/)
- ☐ Brief ธุรกิจ + wireframe
- ☐ Demo สด 15 นาที: เล่าธุรกิจ → โชว์หน้าบ้าน → แก้ข้อมูลในหลังบ้านให้ดูว่าหน้าบ้านเปลี่ยน → ตอบคำถามโค้ด

เกณฑ์ประเมิน (100 คะแนน)

หัวข้อ	คะแนน	ดูอะไร
ฐานข้อมูล	15	schema สมเหตุสมผล, FK/unique ถูก, migration รันได้
Backend / API	25	ครบตาม spec, Zod ทุก endpoint, status code ถูก, auth ปลอดภัย
หน้าบ้าน	20	ครบ 3 หน้า, ดึงจาก API จริง, responsive, loading/error
หลังบ้าน	20	CRUD ครบ, แก้แล้วหน้าบ้านเปลี่ยนจริง, ใช้ง่าย
SEO / Security	10	meta tags, OG, rate limit, cookie ปลอดภัย
Code + README + Demo	10	อ่านง่าย, ไม่มี `any` มั่ว, ตอบคำถามโค้ดตัวเองได้

ผ่าน = 70 คะแนนขึ้นไป และต้องไม่มีข้อต่อไปนี้ (มีข้อใดข้อหนึ่ง = ไม่ผ่านทันที): รหัสผ่านเก็บแบบไม่ hash, admin API เรียกได้โดยไม่ login, โปรเจกต์รันตาม README ไม่ขึ้น

11. Bonus (ทำได้เมื่องานหลักครบแล้วเท่านั้น)

งานหลักยังไม่ครบแต่ไปทำ bonus = ไม่ได้คะแนนเพิ่ม

- อัปโหลดรูปจริง (เก็บไฟล์ในเครื่อง server หรือ S3-compatible) แทนการใส่ URL
- แจ้งเตือนเข้า LINE หรืออีเมลเมื่อมีข้อความติดต่อใหม่
- Prerender หน้า public เพื่อให้ SEO ดีขึ้น
- Docker Compose รัน Postgres + server + client ได้ในคำสั่งเดียว
- Deploy ขึ้น server จริงพร้อมโดเมนย่อย
- เขียน test อย่างน้อยให้ครอบคลุม validator และ auth middleware

Checklist กำหนดงาน (อ้างอิงจาก Timeline 10 วัน + Requirement ด้านบน)

วันที่ 1 - เตรียมโปรเจกต์

☐ เลือกธุรกิจสมมติ 1 ประเภท + เขียน brief (ชื่อแบรนด์/กลุ่มลูกค้า/สี/โลโก้/สินค้าอย่างน้อย 6 รายการ)

☐ ร่าง wireframe 3 หน้า

☐ Setup repo (client+server)

วันที่ 2 - ฐานข้อมูล

☐ ออกแบบ Drizzle schema + รัน migration

☐ เขียน seed script (admin 1, หมวด 2, service 6, settings 1, pages 3)

☐ ทำ ER Diagram

วันที่ 3 - Public API

☐ GET /api/settings, /api/pages/:slug, /api/categories, /api/services

☐ POST /api/contact

☐ Validate ด้วย Zod ทุก endpoint + response format ตรง spec

☐ ทดสอบผ่าน Postman/Httpie

วันที่ 4 - Auth + Admin API

☐ Login/logout + hash password ด้วย Bun.password

☐ JWT เก็บใน httpOnly cookie (sameSite=lax)

☐ Middleware เช็ค token ทุก /api/admin/*

☐ Admin API ครบ: dashboard/settings/pages/services/categories/messages

☐ ทดสอบเรียก admin API โดยไม่ login ต้องได้ 401

วันที่ 5 - Review รอบที่ 1 (โครงสร้าง DB + API กับพี่เลี้ยง)

วันที่ 6 - หน้าบ้าน: Layout + Home

☐ Navbar/Footer/ปุ่ม LINE ลอย

☐ หน้า Home: Hero, บริการเด่น 3 รายการ, จุดเด่นร้าน

☐ เช็ค Responsive 375/768/1280

วันที่ 7 - About/Services + Contact

☐ หน้า About/Services: การ์ดสินค้า + กรองตามหมวด

☐ หน้า Contact: ฟอร์ม + validate + Google Maps embed, ส่งแล้วเข้า DBL

วันที่ 1 - เตรียมโปรเจกต์

ควรศึกษาก่อน: การเขียน brief/wireframe เบื้องต้น, โครงสร้างโฟลเดอร์แบบ monorepo (client/server แยกกัน)

วันที่ 2 - ฐานข้อมูล

ควรศึกษาก่อน: พื้นฐาน SQL (primary key/foreign key), แนวคิด ORM และ migration, syntax เบื้องต้นของ Drizzle schema

วันที่ 3 - Public API

ควรศึกษาก่อน: REST API คืออะไร, HTTP method/status code เบื้องต้น

วันที่ 4 - Auth + Admin API

ควรศึกษาก่อน: JWT คืออะไรทำงานยังไง, httpOnly cookie ต่างจาก localStorage ยังไง, ทำไมต้อง hash password ก่อนเก็บ, concept ของ middleware

วันที่ 5 - Review รอบที่ 1

ควรเตรียม: สรุปปัญหาที่เจอมาให้ชัดก่อนคุยกับพี่เลี้ยง

วันที่ 6 - หน้าบ้าน: Layout + Home

ควรศึกษาก่อน: React component พื้นฐาน, การทำ responsive ด้วย Tailwind breakpoint

วันที่ 7 - About/Services + Contact

ควรศึกษาก่อน: controlled form ใน React, client-side validation เบื้องต้น, วิธี embed Google Maps iframe

วันที่ 8 - Admin หลังบ้าน

ควรศึกษาก่อน: การทำ CRUD UI (list/add/edit/delete), การจัดการ state ของฟอร์มและตาราง

วันที่ 9 - ตรวจความเรียบร้อย + Review รอบที่ 2

ควรศึกษาก่อน: พื้นฐาน SEO (title/meta/OG tag), แนวคิด rate limiting, วิธี handle error ไม่ให้หลุด stack trace

วันที่ 10 - แก้ไข + ส่งงาน

ควรศึกษาก่อน: การเขียน README ที่ดี (setup steps ชัดเจนทำตามได้จริง), การเตรียม demo สั้นๆ ให้ตรงประเด็น